PlacementIQ Enterprise: AI-Powered Placement and Career Readiness System

Shaganti Vivek Goud
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f59@anurag.edu.in

Puram Preethi
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f52@anurag.edu.in

Akshaya Reddy
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f03@anurag.edu.in

Bandari Laxmisri
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
23eg105f09@anurag.edu.in

Dr. Kamalakar Ramineni
Assistant Professor
Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
kamalakar.cse@anurag.edu.in

***Abstract*—Campus placements decide the first career step of most engineering students, yet students rarely know which visiting companies they are eligible for, why they fall short, or what to improve first, and placement cells still screen candidates with CGPA cut-offs shared over spreadsheets and messaging groups. This paper presents PlacementIQ Enterprise, a multi-college placement platform that scores each student's academics, skills, projects, internships, resume and coding-platform activity against the requirements of 66 recruiters and college-posted drives. A transparent multi-criteria engine classifies every student-company pair as Eligible, Nearly Eligible, Can Become Eligible or Not Eligible and states the unmet criteria, while a prediction layer estimates placement readiness, expected salary and interview readiness, and a What-If simulator shows how a planned improvement changes these results. A large language model supports resume analysis, job-description matching and a grounded career assistant, under per-plan usage quotas. Role-based access for students, placement cells, recruiters and platform administrators is enforced with row-level security in the database. Evaluated on 660,000 decisions from 10,000 synthetic student profiles, the engine scored a student against all companies in 0.18 ms on average, explained 100% of non-eligible decisions and showed zero monotonicity violations across 594,000 checks. Compared with cut-off screening, it revealed that 80.3% of cut-off-eligible pairs carry a major skill gap, and the live database passed 19 of 19 access-control tests. The platform turns placement preparation into a measurable, explainable and secure process for students and institutions.**

***Keywords*—Campus placement, eligibility classification, explainable artificial intelligence, skill-gap analysis, large language models, role-based access control**

I. Introduction

Campus recruitment is the main route from an engineering degree to employment in India, and placement outcomes are closely watched by students, institutions and accreditation bodies. A growing body of work applies machine learning to predict whether a student will be placed from academic and demographic records [1]-[3]. These studies confirm that academic performance alone explains only part of the outcome and that skills, projects and internships matter.

At the same time, career-guidance and recommendation systems have matured: skill-assessment tools map students to job roles [4], learning-recommendation systems personalize study paths [5], e-recruitment recommenders match candidates to jobs at scale [6], and large language models (LLMs) are now used to explain and tutor in education [7].

However, these strands rarely meet inside a college's placement process. Prediction models output a single placed or not-placed label that a student cannot act on, explainability remains a recognized weakness of deployed educational models [8], and placement cells still circulate company criteria through portals, spreadsheets, e-mail and messaging groups, screening students by CGPA cut-offs that ignore the skills each company actually tests.

PlacementIQ Enterprise addresses this gap with a single platform in which each college onboards its own students, posts its own drives and monitors every year, branch and section, while each student sees, for every company, an explainable eligibility category, the exact gaps to close, and simulated outcomes of planned improvements.

Contributions of this Paper

- A multi-college placement platform with database-enforced roles for students, placement cells, company recruiters and platform administrators, roster-only student access and per-plan AI usage quotas.
- An explainable multi-criteria eligibility engine that classifies each student against 66 recruiters into four actionable categories and lists every unmet criterion, together with transparent readiness, expected-salary and interview-readiness models and a What-If simulator.
- An LLM-assisted layer for resume analysis (six ATS-style checks), job-description matching and a career assistant grounded in the student's own records.
- A reproducible evaluation on 660,000 synthetic decisions (latency, explanation coverage, monotonicity, comparison with cut-off screening) and a 19-scenario access-control test suite run on the live database.

The rest of the paper is organized as follows. Section II reviews related work, Section III describes the proposed methodology and implementation, Section IV presents the results analysis and comparisons, and Section V concludes the paper with future scope.

A. Motivation

- Every placement season, colleges announce dozens of drives with different CGPA, backlog, branch, batch and skill requirements; students must interpret each one manually and often discover a missing requirement only after rejection.
- Existing practice is scattered across placement portals, spreadsheets, e-mail and WhatsApp groups, and screening by CGPA cut-off says nothing about whether a student can clear the coding, design or aptitude rounds.
- An explainable, always-current view of eligibility saves placement officers repetitive filtering, gives students a concrete preparation plan months before a drive, and lets institutions monitor readiness by year, branch and section.
- The project was driven by the authors' own placement preparation at Anurag University and by the department's need for a data-driven view of student readiness across batches.

B. Problem Statement

Given a student profile x (CGPA, backlogs, Class X and XII scores, eight skill levels, projects, internships, resume and coding-platform statistics) and a set of companies C, each with requirements r_c, the goal is to assign every pair (x, c) to one of four ordered categories with an explicit list of unmet requirements, estimate placement readiness, expected salary and interview readiness, and expose all results only to the users entitled to see them.

- Classify a student against all companies in under 1 ms of computation, so that the dashboard and What-If simulator update instantly.
- Attach at least one concrete, unmet criterion or skill gap to 100% of non-eligible decisions, and guarantee that an improvement in any input never lowers a match score or category (monotonicity).
- Isolate data between colleges and roles so that 100% of defined access-control scenarios pass on the production database.

II. Literature Survey (Papers from IEEE, ACM, and Springer)

Placement and employability prediction. Basha et al. combined exploratory analysis with supervised classifiers to predict campus placement success [1], Krishnaiah and Kadegowda proposed a hybrid machine-learning approach for predicting the employment of undergraduate engineering students [2], and Kunwar et al. used a CNN-BiLSTM network with attention to predict placement and recommend skills [3]. These models report useful predictive power, but they produce a single outcome per student rather than a per-company decision, and deep models such as [3] are difficult to explain. A systematic review of student-performance prediction found that explainability is still rarely achieved in practice [8].

Career guidance and learning recommendation. C3-IoC relates student skills to IT job roles through a similarity metric and network visualization to support career exploration [4]. Reviews of personalized learning recommendation summarize learner modelling and recommendation algorithms for e-learning [5]. LLMs bring new opportunities and risks to education [7], and recent work mines user intents with LLMs to build career-guidance chatbots [9]. These systems advise on careers in general, but they are not connected to the concrete criteria of drives visiting a particular college.

Person-job fit, e-recruitment and resume matching. Neural person-job fit models learn the match between candidate abilities and job requirements from recruitment histories [10], and two-way selection models capture the preferences of both candidates and recruiters [11]. A challenge-based survey of e-recruitment recommenders highlights data sparsity, explainability and fairness as open problems [6]. On the text side, BERT [13] and Sentence-BERT [14] embeddings enable semantic resume and job-description matching and ranking [12], and annotated corpora such as SkillSpan support hard- and soft-skill extraction from job postings [15]. These works mostly serve recruiters, require large labelled hiring histories, and do not give students actionable feedback.

Evidence from developer platforms. Developer expertise can be inferred from GitHub repositories with natural-language processing [16] and from the GitHub social network with graph neural networks [17], which motivates using verified coding-platform activity instead of self-reported skills alone.

A. Comparative Summary of Related Work

Table I compares representative studies by year, method, data and main limitation. Most of them predict a single placement outcome, recommend careers or learning in general, or rank candidates for recruiters; none of them tells a student, for each visiting company, whether they are eligible, why not, and what to improve.

TABLE I
COMPARISON OF EXISTING APPROACHES

Ref.	Year	Method	Dataset	Limitation
[1]	2023	EDA + supervised ML classifiers	Institutional placement records	Placed or not only; no per-company criteria
[2]	2022	Hybrid machine learning	Engineering graduate records	Outcome prediction; no improvement plan
[3]	2025	CNN-BiLSTM with attention	Student academic and skill data	Black-box model; weak explanation
[4]	2023	Skill similarity + job-role network	IT job roles, student skills	Career exploration, not campus drives
[5]	2024	Review of learning recommenders	Studies 2013-2023	Learning-centric; no recruiter criteria
[6]	2024	Survey of e-recruitment recommenders	Published systems	Explainability and fairness open
[8]	2021	Review of explainable prediction	Studies 2015-2020	Few deployed explainable models
[9]	2025	LLM intent mining for chatbots	Career-guidance queries	Not grounded in student records
[10]	2018	Ability-aware neural person-job fit	Recruitment histories	Needs large hiring data; recruiter-side
[12]	2023	NLP and BERT resume ranking	Resumes, job descriptions	Ranks candidates; no student feedback
[15]	2022	Skill extraction corpus	English job postings	Extraction only; no matching to students
[17]	2024	GNN on GitHub social network	GitHub developers	Expertise only; no placement context

B. Research Gap

Existing work either predicts a single placement outcome from academic records, recommends careers or learning paths without reference to the drives visiting a college, or ranks candidates for recruiters. None of the reviewed systems gives each student an explainable, per-company eligibility decision with the exact gaps and a way to test improvements, and none combines this with multi-college data isolation and role-specific views for placement cells and recruiters. PlacementIQ Enterprise fills this gap with a transparent eligibility engine, explainable predictions, a What-If simulator, LLM-assisted resume tools and database-enforced multi-tenant access control.

III. PROPOSED METHODOLOGY / IMPLEMENTATION

A. System Overview

Fig. 1 shows the architecture. Users of four roles reach a React single-page application that routes each role to its own pages. The eligibility engine, prediction layer and What-If simulator run in the browser on the student's own data, so results update without a server round trip. Data lives in Supabase (PostgreSQL), where row-level security policies, the role model of [18] and private helper functions decide which rows each request may read or change [19]. Serverless edge functions hold the trusted keys and implement resume analysis, job-description matching, the career assistant (through the gpt-oss-120b model [20]), coding-profile synchronization, roster sign-up and administrative invitations.

A platform administrator creates colleges and their placement-cell logins; each placement cell uploads its roster (e-mail, name, roll number, branch, graduation year, section), posts drives and adds company recruiters. Only e-mail addresses registered by these roles can hold an account; this is enforced by a trigger on the authentication table.

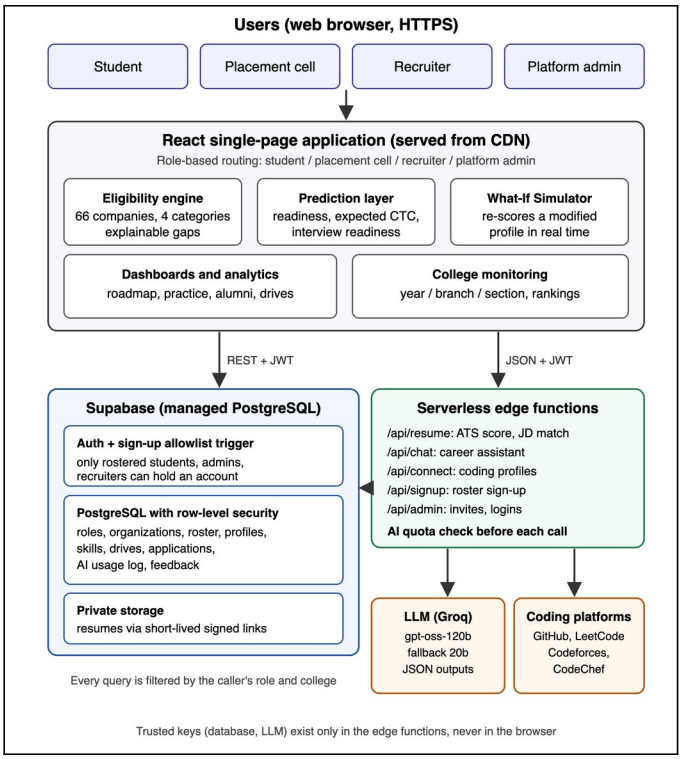


Fig. 1. Proposed system architecture of PlacementIQ Enterprise.

B. Dataset / Data Collection

PlacementIQ does not train a model on a fixed dataset; it scores live records. Student data comes from mandatory onboarding and the profile: CGPA, active backlogs, Class X and XII percentages, levels (0-100) for eight skills (data structures and algorithms, system design, frontend, backend, databases, machine learning, cloud and DevOps, aptitude), projects, internships, certifications and achievements. Resumes are parsed in the browser (PDF with pdf.js, DOCX with mammoth); coding statistics are fetched from the public GitHub, LeetCode, Codeforces and CodeChef interfaces.

Company data covers 66 recruiters grouped into ten categories (for example top product, quant, AI, fintech, startup, IT services). Each category defines minimum CGPA, maximum backlogs, Class X and XII minimums, internships, projects and eight skill levels, with a small deterministic per-company variation; drives posted by a placement cell or recruiter carry their own explicit criteria and job description. Skill levels are normalized to 0-100, CGPA to 0-10 and percentages to 0-100.

For evaluation, a cohort of 10,000 synthetic students was generated with a fixed-seed random generator: CGPA 5.5-9.8, 82% without backlogs, a latent ability that drives correlated skill levels, 0-5 projects and 0-2 internships. The fixed seed makes every reported number reproducible.

C. Algorithm / Model Design

For a student x and company c with requirements r_c, the engine first computes an academic score A as the mean of four capped ratios of CGPA g, Class X score X and Class XII score Y to their minimums (starred), where b = 1 if the backlogs are within the limit and 0.4 otherwise:

$$A = 25\,[\hat{g} + X + \hat{Y} + b],\ \ \hat{g} = min(1, g/g^{*}) \tag{1}$$

Each skill k with required level r_k and student level s_k contributes its shortfall, so several small gaps add up the way they do in a multi-round interview:

$$S = max(0,\ 100 - 1.2\,\Sigma_k\ max(0,\ r_k - s_k)) \tag{2}$$

With experience E and project score P (capped ratios to the required internships and projects, counting original GitHub repositories as projects), the match score is:

$$M = 0.15\,A + 0.60\,S + 0.10\,E + 0.15\,P \tag{3}$$

Categories are assigned in order. If an academic minimum fails, the pair is Can Become Eligible when the student is within 0.5 CGPA and 5 percentage points of every minimum and $M \geq 55$, and Not Eligible otherwise. Otherwise it is Eligible if $M \geq 88$, the largest gap $\Delta max \leq 5$ and internships are met; Nearly Eligible if $M \geq 75$, $\Delta max \leq 15$ and the total gap ≤ 25; Can Become Eligible if $M \geq 55$; and Not Eligible otherwise. Every decision stores its criteria and sorted gaps, which are shown to the student. Placement readiness is the mean match over the target categories T (top product, product, startup and fintech):

$$R = (1/|T|)\,\Sigma_{(c \in T)}\,M_c \tag{4}$$

Expected salary weights each company's average package by an offer likelihood $w_c = \omega(category_c) \cdot M_c / 100$, with ω = 0.55, 0.30, 0.10 and 0.01 for the four categories; the 25th and 75th weighted percentiles give the likely band:

$$\hat{CTC} = \Sigma_c\ w_c\ CTC_c\ /\ \Sigma_c\ w_c \tag{5}$$

Interview readiness combines coding skill D, aptitude T, system design Z, the mean logged mock-interview score K (out of 10, scaled to 100), resume score Q and experience V (20 per project and 25 per internship, capped at 100):

$$I = 0.3D + 0.15T + 0.15Z + 0.2K + 0.1Q + 0.1V \tag{6}$$

Placement cells rank students with a composite score that also uses profile completeness U and the CGPA term G = min(100, 10g):

$$Score = 0.35R + 0.2Q + 0.2D + 0.15U + 0.1G \tag{7}$$

The What-If simulator applies the same functions to a modified copy of the profile (CGPA, backlogs, skill levels, extra projects or internships, mock scores), lists the companies whose category changes, and tests a +10 step on each skill to report the single most effective improvement. LLM features (resume analysis with six ATS checks, job-description matching, the career assistant) receive the structured profile as context, return JSON or grounded text, and each call first consumes one action from the student's monthly plan quota.

D. Implementation Details

The front end uses React 18 with TypeScript, Vite 5 and Tailwind CSS, with Recharts for analytics and lazily loaded three.js for decorative 3D on the landing page. The back end is Supabase (PostgreSQL with row-level security, authentication and private storage) and Vercel serverless edge functions. The LLM is gpt-oss-120b served by Groq, with gpt-oss-20b as fallback. Roles (super_admin, org_admin, recruiter) are stored in a user_roles table and checked by SECURITY DEFINER helper functions in a private schema; AI quotas are 15, 40, 80 and 150 actions per student per month for the Trial, Basic, Pro and Enterprise plans. The evaluation scripts ran on Node.js 25 on an Apple Silicon laptop.

E. Evaluation Metrics

Because no labelled placement outcomes are available yet, the system is evaluated on properties that its design requires: (i) latency per student for all 66 companies and all predictions (mean and 95th percentile); (ii) explanation coverage, the share of non-eligible decisions that state at least one unmet criterion or gap; (iii) monotonicity, the share of checks in which raising one skill by 10 or CGPA by 0.5 never lowers a match score or category; (iv) divergence from CGPA/backlog cut-off screening; (v) What-If actionability, the share of students for whom one +10 skill step moves at least one company up a category; and (vi) the pass rate of access-control tests.

IV. Results Analysis and Comparisons

A. Experimental Setup

The engine and prediction code of the deployed application were bundled unchanged into an evaluation script (docs/evaluation/evaluate.ts in the project repository). It scored 10,000 synthetic students against 66 companies (660,000 decisions), ran 594,000 monotonicity checks on 1,000 students, and compared every decision with a baseline that accepts a student when CGPA and backlogs meet the company minimum. Access control was tested with 19 scenarios on the production database inside a transaction that was rolled back, so no data was changed.

B. Performance Results

Table II lists the seven measurements E1 to E7 with the result, the number of samples and the outcome of each. Fig. 2 compares CGPA cut-off screening with the proposed multi-criteria classification over the same 660,000 student-company decisions, and Fig. 3 shows how one decision and its reasons appear to a student in the deployed application.

TABLE II
MEASURED PERFORMANCE OF THE PROPOSED SYSTEM

No.	Metric	Result	Samples	Outcome
E1	Latency per student, mean / p95	0.18 / 0.25 ms	10,000 students	Real time
E2	Non-eligible decisions with a stated reason	100%	629,068 decisions	Pass
E3	Monotonicity violations	0	594,000 checks	Pass
E4	Cut-off accepts with a major skill gap	80.3%	341,912 pairs	Baseline over-admits
E5	Cut-off rejects that can become eligible	10.3%	318,088 pairs	Baseline gives no path
E6	One +10 skill step moves a company up	36.7%	1,000 students	Actionable
E7	Access-control tests passed	19 / 19	19 scenarios	Pass

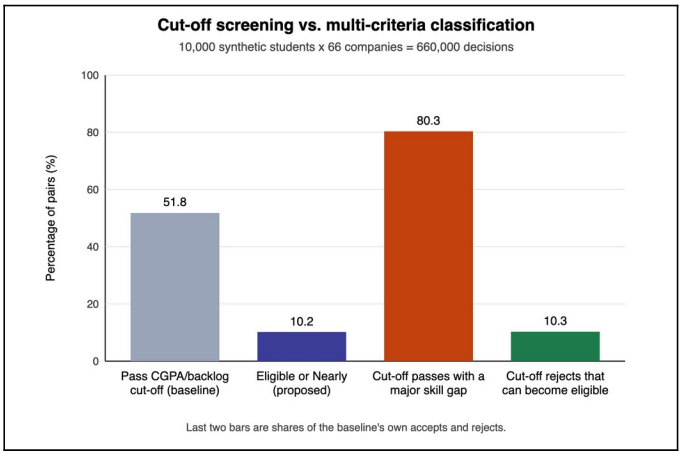


Fig. 2. Cut-off screening compared with the proposed multi-criteria classification (660,000 decisions).

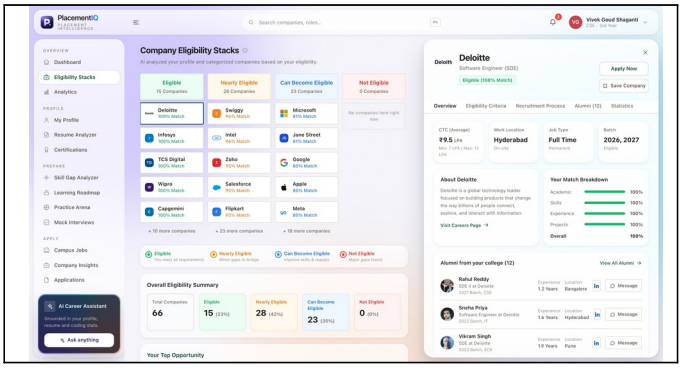


Fig. 3. Company Eligibility Stacks screen with a company panel showing criteria and match breakdown (sample data).

C. Discussion

Table II and Fig. 2 summarize the results. All predictions for one student across 66 companies took 0.18 ms on average (0.25 ms at the 95th percentile), well inside the 1 ms objective, which is why the dashboard and the What-If simulator respond instantly in the browser. Every one of the 629,068 non-eligible decisions named at least one unmet criterion or skill gap, and no improvement in any input ever lowered a match or category in 594,000 checks, so students are never penalized for improving.

The comparison with cut-off screening is the main practical finding. The cut-off accepted 51.8% of all student-company pairs, but 80.3% of those accepted pairs had at least one required skill more than 15 points short or were classified Can Become or Not Eligible, so cut-off lists send many unprepared candidates into drives. Conversely, 10.3% of the pairs rejected by the cut-off were close enough to be Can Become Eligible, a group the cut-off gives no guidance at all. For 36.7% of students a single 10-point improvement in one skill moved at least one company up a category, which the What-If simulator surfaces as the most effective next step. All 19 access-control scenarios passed, covering the sign-up allowlist, roll-number verification, isolation between colleges, recruiter restrictions and the AI quota.

Unlike the predictors in [1]-[3], the proposed engine outputs a decision and a reason per company, and unlike the recruiter-side matchers in [10]-[12] it is designed to guide students. The trade-off is that its thresholds and category weights are expert-defined rather than learned. The evaluation uses a synthetic cohort and checks design properties, not agreement with real placement outcomes, so no accuracy is claimed; company requirement templates are approximations of public hiring patterns; and LLM outputs for resume analysis and the assistant were not benchmarked quantitatively. These limitations define the pilot study planned as future work.

V. CONCLUSION AND FUTURE SCOPE

A. Conclusion

This paper presented PlacementIQ Enterprise, a multi-college placement platform that replaces CGPA cut-off screening and scattered communication with explainable, per-company eligibility for every student. A transparent engine classified 660,000 student-company pairs in well under a millisecond per student, explained every non-eligible decision, and never penalized an improvement; it showed that 80.3% of cut-off-eligible pairs hide a major skill gap. Readiness, expected-salary and interview-readiness estimates, a What-If simulator and LLM-assisted resume tools turn these decisions into a preparation plan, while row-level security keeps each college's and recruiter's data isolated, as confirmed by 19 of 19 access-control tests.

B. Future Scope

- Run a pilot across all batches at Anurag University, collect actual placement outcomes, and use them to calibrate the thresholds and to train and validate a learned model (for example gradient boosting with SHAP explanations) against the transparent engine.
- Extend the recruiter and placement-cell workflow with interview scheduling, offer tracking, faculty mentor roles, single sign-on and analytics across colleges.
- Scale the deployment on AWS with load testing, fairness audits of the eligibility rules and data-retention and consent

controls aligned with India's Digital Personal Data Protection Act, 2023.

ACKNOWLEDGMENT

The authors thank their guide, Dr. Kamalakar Ramineni, Assistant Professor, Department of Computer Science and Engineering, Anurag University, for his guidance throughout this project, and the department for its support and facilities.

REFERENCES

[1] M. S. A. Basha, R. P. K. Raju, P. M. Oveis, and M. Sarim, "Unraveling campus placement success integrating exploratory insights with predictive machine learning models," in *Proc. 7th Int. Conf. Comput. Syst. Inf. Technol. Sustain. Solutions (CSITSS)*, Bengaluru, India, Nov. 2023, pp. 1-6, doi: 10.1109/CSITSS60515.2023.10334118.

[2] V. Krishnaiah and Y. H. Kadegowda, "Undergraduate engineering students employment prediction using hybrid approach in machine learning," *Int. J. Elect. Comput. Eng.*, vol. 12, no. 3, pp. 2783-2791, Jun. 2022, doi: 10.11591/ijece.v12i3.pp2783-2791.

[3] F. B. Kunwar, A. Ramachandran, E. Manigandan, S. Srivastava, S. Priya, and S. Suhane, "Enhanced student placement prediction and skill recommendation system using convolutional neural network-bidirectional long short-term memory with attention networks," in *Proc. 7th Int. Conf. Intell. Sustain. Syst. (ICISS)*, 2025, pp. 1450-1456, doi: 10.1109/ICISS63372.2025.11076538.

[4] A. José-García et al., "C3-IoC: A career guidance system for assessing student skills using machine learning and network visualisation," *Int. J. Artif. Intell. Educ.*, vol. 33, no. 4, pp. 1094-1121, 2023, doi: 10.1007/s40593-022-00317-y.

[5] Q. Bin, M. F. Zuhairi, and J. Morcos, "A comprehensive study on personalized learning recommendation in e-learning system," *IEEE Access*, vol. 12, pp. 100446-100482, 2024, doi: 10.1109/ACCESS.2024.3428419.

[6] Y. Mashayekhi, N. Li, B. Kang, J. Lijffijt, and T. De Bie, "A challenge-based survey of e-recruitment recommendation systems," *ACM Comput. Surv.*, vol. 56, no. 10, pp. 1-33, Oct. 2024, doi: 10.1145/3659942.

[7] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individ. Differ.*, vol. 103, Art. no. 102274, 2023, doi: 10.1016/j.lindif.2023.102274.

[8] R. Alamri and B. Alharbi, "Explainable student performance prediction models: A systematic review," *IEEE Access*, vol. 9, pp. 33132-33143, 2021, doi: 10.1109/ACCESS.2021.3061368.

[9] M. Bhatnagar, "Enhancing career guidance through intent mining with large language models," in *Artificial Intelligence in Education Technologies: New Development and Innovative Practices* (Lecture Notes on Data Engineering and Communications Technologies), T. Schlippe, E. C. K. Cheng, and T. Wang, Eds. Singapore: Springer, 2025, pp. 17-33, doi: 10.1007/978-981-97-9255-9_2.

[10] C. Qin et al., "Enhancing person-job fit for talent recruitment: An ability-aware neural network approach," in *Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, Ann Arbor, MI, USA, 2018, pp. 25-34, doi: 10.1145/3209978.3210025.

[11] C. Yang, Y. Hou, Y. Song, T. Zhang, J.-R. Wen, and W. X. Zhao, "Modeling two-way selection preference for person-job fit," in *Proc. 16th ACM Conf. Recommender Syst. (RecSys)*, Seattle, WA, USA, 2022, pp. 102-112, doi: 10.1145/3523227.3546752.

[12] S. Tanberk, S. S. Helli, E. Kesim, and S. N. Cavsak, "Resume matching framework via ranking and sorting using NLP and deep learning," in *Proc. 8th Int. Conf. Comput. Sci. Eng. (UBMK)*, Burdur, Türkiye, Sep. 2023, pp. 453-458, doi: 10.1109/UBMK59864.2023.10286605.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171-4186, doi: 10.18653/v1/N19-1423.

[14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, Hong Kong, China, 2019, pp. 3980-3990, doi: 10.18653/v1/D19-1410.

[15] M. Zhang, K. Jensen, S. Sonniks, and B. Plank, "SkillSpan: Hard and soft skill extraction from English job postings," in *Proc. NAACL-HLT*, Seattle, WA, USA, 2022, pp. 4962-4984, doi: 10.18653/v1/2022.naacl-main.366.

[16] S. Kourtzanidis, A. Chatzigeorgiou, and A. Ampatzoglou, "RepoSkillMiner: Identifying software expertise from GitHub repositories using natural language processing," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2020, pp. 1353-1357, doi: 10.1145/3324884.3415305.

[17] Y. Sun et al., "Automatically deriving developers' technical expertise from the GitHub social network," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, 2024, pp. 2462-2463, doi: 10.1145/3691620.3695329.

[18] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38-47, Feb. 1996, doi: 10.1109/2.485845.

[19] Supabase, "Row Level Security." Supabase Docs. https://supabase.com/docs/guides/database/postgres/row-level-security (accessed Oct. 6, 2026).

[20] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," *arXiv*:2508.10925, Aug. 2025, doi: 10.48550/arXiv.2508.10925.